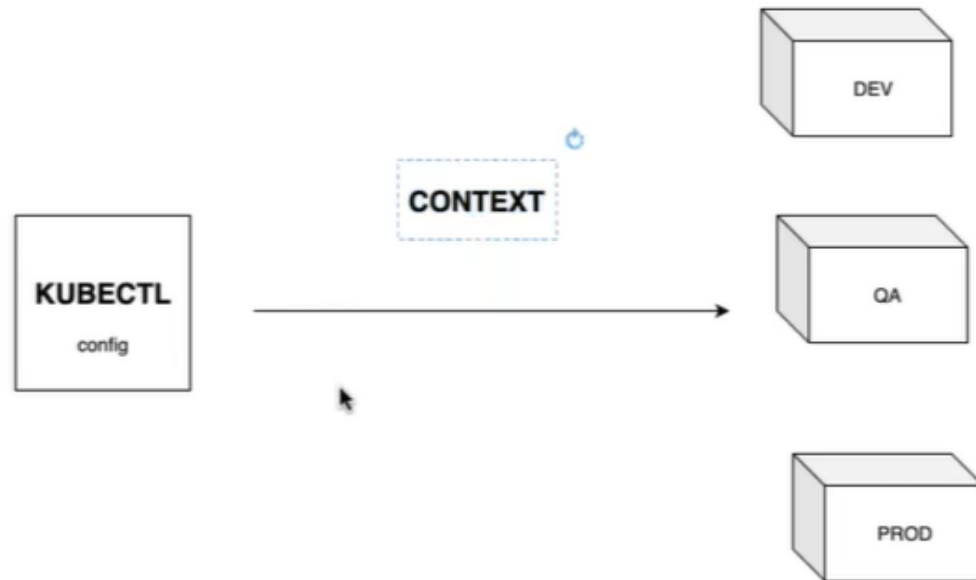


UC: In your organization there are multiple cluster

Jump Box/Bastion host

What required in Jump Box?

kubectl +. kubeconfig



step1: create a new instance jumpbox.

step2: Install kubectl

`sudo apt-get install -y kubectl` (only kubectl client utility)

step3: create devops user (password today@1234)

step4: create devops user (password today@1234) in master

step5: `cp .kube/config /tmp; chmod 755 /tmp/config`

step6: scp of .config from existing master

`scp devops@<master> :/tmp/config .`

step7: `mkdir .kube`

step8: `mv config .kube/`

step9: `kubectl get nodes`

```
root@test:~# scp devops@10.128.0.46:/tmp/config .
The authenticity of host '10.128.0.46 (10.128.0.46)' can't be established.
ECDSA key fingerprint is SHA256:uPKje9lP8Dps3atlxFbr8HC63DUBQwJLR2Fvjpt50o4.
Are you sure you want to continue connecting (yes/no)? yes
Warning: Permanently added '10.128.0.46' (ECDSA) to the list of known hosts.
devops@10.128.0.46's password:
config
root@test:~# mkdir .kube
root@test:~# mv config .kube/
root@test:~# kubectl get nodes
NAME          STATUS    ROLES    AGE   VERSION
masternode    Ready     master   16d   v1.19.0
node1         Ready     <none>    16d   v1.19.0
node2         Ready     <none>    16d   v1.19.0
root@test:~#
```

how to create a user

```
mkdir -p /home/prod-user/certs  
cd /home/prod-user/certs
```

generate private key

```
openssl genrsa -out prod-user.key 2048
```

create certificate signing request (csr) file

```
openssl req -new -key prod-user.key -out prod-user.csr # common name,  
organization name
```

request is the base64 encoded value of the CSR file content. You can get the content using this command:

```
cat prod-user.csr | base64 | tr -d "\n"
```

create csr manifests

```
nano prod-user.yaml
```

```
apiVersion: certificates.k8s.io/v1  
kind: CertificateSigningRequest  
metadata:  
  name: prod-user  
spec:  
  groups:  
    - system:authenticated  
  request: cat prod-user.csr | base64 | tr -d "\n"  
  signerName: kubernetes.io/kube-apiserver-client  
  usages:  
    - client auth
```

```
kubectl create -f prod-user.yaml
```

```
kubectl get csr
```

```
# pending approval
```

```
kubectl certificate approve prod-user
```

```
# csr approved
```

how user use this certificate

```
sudo cp /etc/kubernetes/admin.conf .  
mv admin.conf prod-user.conf
```

Update the prod-user.conf

.kubeconfig Kubeconfig file is in specific format, config file has three sections
Cluster, User and Context

```
kind: Config
apiVersion: v1
preferences: {} # current-context
clusters:
users:
contexts:
```

```
kind: Config
apiVersion: v1
clusters:
  - name:
    cluster:
users:
  - name:
    user:
contexts:
  - name:
    context:
```

```
kind: Config
apiVersion: v1
clusters:
  - name: k8s
    cluster:
      certificate-authority-data: # same
      server: https://127.0.0.1:6443 # same api server
users:
  - name: prod-user # update
    user:
      client-certificate-data / client-certificate: # -data remove exact path
      client-key-data / client-key: # -data remove exact path
contexts:
  - name: prod-user-k8s
    context:
      cluster: k8s
      user: prod-user
      namespace: # dev
```

commands

```
k config -h
k config view # default config from /.kube
k config --kubeconfig=/my-custom-config view # custom config
k config get-contexts # displays list of the context
k config current-context # display the current-context
k config set-context --current --namespace=dev # permanently save the
namespace for all subsequent kubectl commands in that context.
k config unset users.foo # delete user foo
k config use-context prod-k8s-playground@prod-k8s-admin
k config use-context prod-k8s-playground@prod-k8s-admin --kubeconfig=/my-
custom-config
```

Once we move our custom-config in to .kube/ folder then it becomes our default config

validate

```
kubectl --kubeconfig=prod-user.conf version --short
kubectl --kubeconfig=prod-user.conf get nodes # As of now authenticated not
authorized
```

UC: we have multiple clusters, how to connect to the Dev?

Using Context

```
kubectl config get-contexts
kubectl config use-context <name>
```

Context binds your kubectl with your cluster
Context tells you which user is connected to which cluster